2026 年“华为杯”华南理工大学程序设计竞赛（秋季赛）（第二场）

SCUTPC 2026 出题组

SCUT

2026 年秋季

目录

1. 总体介绍
2. Problem A：蒲公英计数
3. Problem B：Shelter
4. Problem C：GCD
5. Problem D：走地鸡的 RPG
6. Problem E：春日写生
7. Problem F：求王北伐
8. Problem G：绝趋零
9. Problem H：SCUT
10. Problem I：随机加减
11. Problem J：暴铺
12. Problem K：小 Z 的小 Z
13. Problem L：Cinderella
14. Problem M：xboHodx 是回文串
15. Problem N：与或异或
16. Problem O：十夏之花

难度预估

难度	题号
Easy	B H I
Medium-Easy	C K
Medium	D E L
Medium-Hard	F G N
Hard	A J M
Impossible	O

知识点分布

题号	知识点	题号	知识点
A	圆方树，换根 dp	H	贪心
B	推式子	I	期望线性性
C	辗转相除	J	贪心，dp
D	三分	K	思维，欧拉公式
E	单调栈	L	扫描线
F	kruskal 重构树/dijkstra 瓶颈路	M	分层图最短路
G	线段树	N	位运算拆位
		O	多项式 exp，多项式求根

题目大意

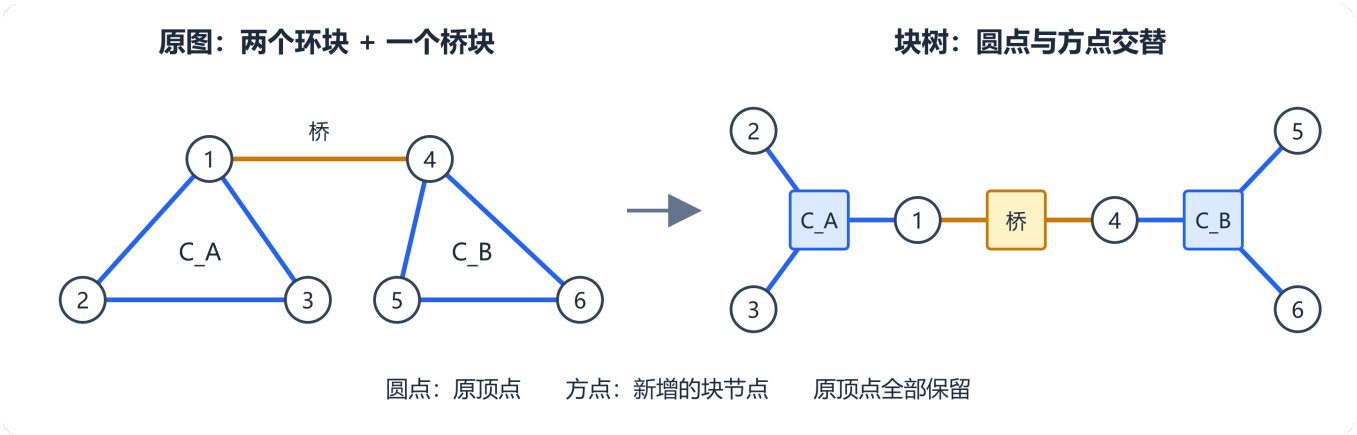
A 蒲公英计数

——By welken

给定仙人掌图：无向简单图，每条边至多属于一个简单环，可以不连通。
求其中有多少个子图是一棵蒲公英，即“一个简单环，挂着一条非空简单路径”，环与路径只共享路径的一个端点。
两个子图不同，当且仅当选取的边集不同。可以不选原图中的其他边，答案对 998244353 取模。
数据范围：$T \le 10^4$，$\sum n, \sum m \le 2 \times 10^5$。

块树：把仙人掌变成森林

仙人掌由两种基本结构拼成：一条桥，或一个简单环，称它们为“块”。蒲公英的花一定是一个环块，桥无法成为花。
保留所有原顶点，为每个块新建一个方形节点，向它包含的原顶点连边，得到一片森林，称为块树。块由 Tarjan 求点双连通分量得到，原顶点编号 $1 \sim n$，块节点另编新号。



块树上任意两点之间只有一条路。因此一条链经过的块顺序确定后，只需考虑如何穿过每个块：桥只有一条路，环上两点之间有两条弧。需要快速判断块是桥还是环。

状态：按链尾计数

从花出发，把链延伸到顶点 v，再按链尾 v 统计。合法蒲公英的链尾唯一，这样不会重复计数。

为统一转移，暂时允许链长为 0：一个环配上环上的一个指定顶点，也算一个临时方案。

在块树上，一条块树边 vB 两侧各有一个数：

- $msg_{B\to v}$：块 B 这一侧、以 v 为链尾的方案数，也就是 B 传给 v 的消息；
- $acc_{v\to B}$：顶点 v 从除 B 以外的各个块收到的消息之和。

“这一侧”指删去块树边 vB 后对应的部分。方案可以延伸到接口 v，但不能绕回另一侧。

顶点只汇总除 B 之外的块：

$$acc_{v\to B}=\sum_{B'\ni v,\ B'\neq B} msg_{B'\to v}.$$

对于桥块 $B=\{u,v\}$，把以 u 为链尾的方案沿桥延伸到 v：

$$msg_{B\to v}=acc_{u\to B}.$$

环块转移：两条弧

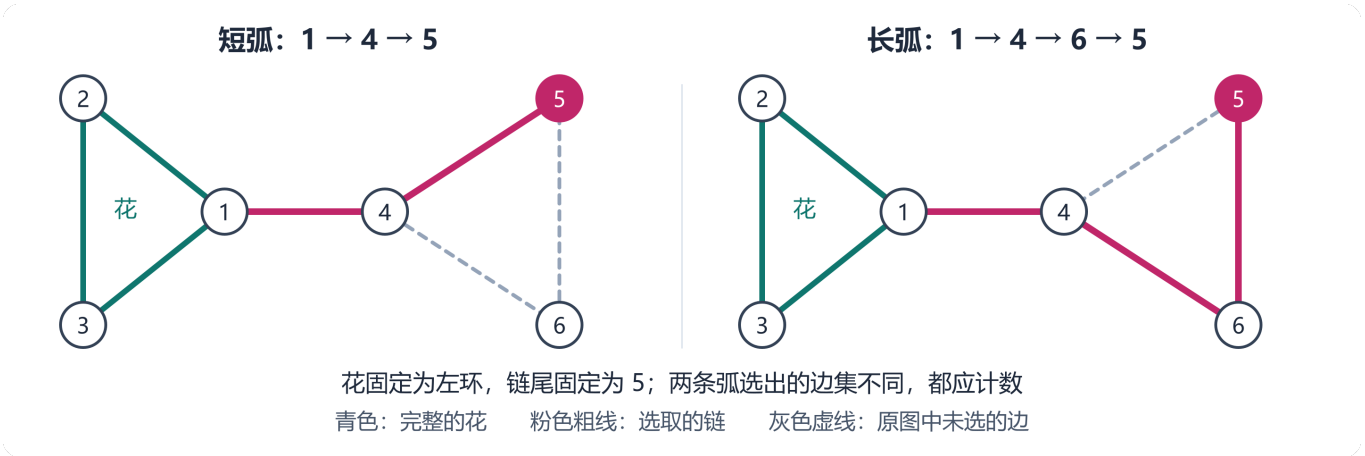
对于环块 $C=\{v_0,\dots,v_{k-1}\}$，记 $acc_i=acc_{v_i\to C}$，$sum=\sum_j acc_j$。
要把链尾延伸到 v_i，有两类方案：

- 花就是环 C，链长为 0：1 种；
- 花在其他块中，链从 v_j 进入 C $(j\neq i)$，沿两条弧之一走到 v_i：$2acc_j$ 种。

因此

$$msg_{C\to v_i}=1+2(sum-acc_i).$$

先算出 sum，每个点的转移就是 $O(1)$，不需要枚举点对。



花固定为左环，链尾固定为 5；两条弧选出的边集不同，都应计数
青色：完整的花　粉色粗线：选取的链　灰色虚线：原图中未选的边

未选中的边不会影响子图是否合法。

两遍树上 DP

块树是森林，在每棵树上任取一个顶点作根，做两遍扫描。
第一遍自底向上，只算子树一侧。顶点把子块的消息累加进 acc；块把自己的消息存进 $down$，同时加到父顶点的 acc 上。设块 B 的父顶点是 p：

$$down_B = \begin{cases} acc_u, & B = \{p, u\} \text{ (bridge)}, \\ 1 + 2\sum_u acc_u, & B \text{ (cycle)}. \end{cases}$$

式中 acc_u 即 $acc_{u \to B}$，求和只跑 B 的子顶点，父顶点 p 不参与。这里不需要父侧的数据：式子里被排除的恰好就是 p，而 $\sum_{u \neq p} acc_u$ 在自底向上时已经齐了。
第二遍自顶向下，补齐父侧。对顶点 u 的每个子块 B，由

$$acc_{u \to B} = total_u - down_B$$

反解出父侧那一份：顶点 u 收到的全部消息，减去 B 自己那一条。于是环内完整的总和是

$$sum = acc_{u \to B} + \sum_x acc_{x \to B},$$

其中 x 跑 B 的子顶点。再对每个 x 算出消息并写进 $total$：

$$total_x = acc_{x \to B} + \begin{cases} acc_{u \to B}, & B \text{ (bridge)}, \\ 1 + 2(sum - acc_{x \to B}), & B \text{ (cycle)}. \end{cases}$$

答案与复杂度

两遍树形 dp 之后

$$total_v = \sum_{B \ni v} msg_{B \to v}$$

就是以 v 为链尾的方案数，包含临时允许的空链。

$$\text{Ans} = \sum_{v=1}^{n} total_v - \sum_{C} |C| \pmod{998244353},$$

其中 C 取遍所有环块。一个长度为 k 的环，在每个环上顶点各产生一个空链方案，需要扣掉 k 个。
总时间复杂度 $O(\sum(n+m))$，单组空间复杂度 $O(n+m)$。

题目大意

B Shelter

——By XiaoXia

求

$$\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{k=1}^{n} ijk$$

对 988244353 **取模后的结果。**

式子化简

移项化简式子即可：

$$\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{k=1}^{n} ijk = \sum_{i=1}^{n} i \sum_{j=1}^{n} j \sum_{k=1}^{n} k = (\sum_{i=1}^{n} i)^3 = (\frac{n(n+1)}{2})^3$$

需要注意是对 988244353 取模而不是 998244353。
时间复杂度：$\mathcal{O}(1)$。

题目大意

C GCD

——By Crsuh2er0

给定三个正整数 a, x, y，定义

$$\mathrm{GCD}(x, y) = \gcd\left(a^x - 1,\ a^y - 1\right),$$

求 $\mathrm{GCD}(x, y)$ 对 998244353 取模的结果，其中 $\gcd$ 表示最大公约数。

解析

答案：$\mathrm{GCD}(x,y)=\gcd(a^x-1,\ a^y-1)=a^{\gcd(x,y)}-1$，**故输出** $\left(a^{\gcd(x,y)}-1\right) \bmod 998244353$。

证明

首先 $\mathrm{GCD}(x,y)=\gcd(a^x-1,\ a^y-1)$.
因为 $\mathrm{GCD}(x,y)=\mathrm{GCD}(y,x)$，**不妨设** $x\geq y$，**则有**
$\gcd(a^x-1,\ a^y-1)=\gcd(a^y-1,a^x-1 \bmod a^y-1)$.
由于 $a^x-1\equiv a^x-1-a^{x-y}(a^y-1)\equiv a^x-1-a^x+a^{x-y}\equiv a^{x-y}-1 \pmod{a^y-1}$,
反复应用上述同余关系可得 $a^x-1\equiv a^{x \bmod y}-1 \pmod{a^y-1}$,
所以 $\gcd(a^x-1,a^y-1)=\gcd(a^y-1,a^{x \bmod y}-1)$.
即 $\mathrm{GCD}(x,y)=\mathrm{GCD}(y,x \bmod y)$.
每一步，(x,y) **变为** $(y,x \bmod y)$，**这与求** $\gcd(x,y)$ **的过程是相同的。**
所以 (x,y) **最终变为** $(\gcd(x,y),0)$.
故 $\mathrm{GCD}(x,y)=\mathrm{GCD}(\gcd(x,y),0)=\gcd(a^{\gcd(x,y)}-1,0)=a^{\gcd(x,y)}-1$. ■

常见错误：

- **快速幂底数未取模**：计算 $a^{\gcd(x,y)} \bmod 998244353$ 时，应使用 `a % MOD` 作为底数，避免乘法溢出。
- **输出负数**：答案为 $(a^{\gcd(x,y)}-1) \bmod 998244353$，若 $998244353 \mid a$，快速幂结果为 0，直接减一会得到负数，应写成 `(ans - 1 + MOD) % MOD`。

复杂度：时间 $O(\log x)$，空间 $O(1)$。

题目大意

D 走地鸡的 RPG

——By Crsuh2er0

设分配给 A 资源用于提升暴击率的数量为 x（整数，$0 \le x \le n_A$），分配给 B 资源用于提升暴击率的数量为 y（整数，$0 \le y \le n_B$）。则分配后属性为：

$$r = r_0 + a_r x + b_r y,$$
$$k = k_0 + a_k(n_A - x) + b_k(n_B - y).$$

最大化 $D = 1 + r(k-1)$。

问题转化

设分配给 A 资源加暴击率的份数为整数 x $(0 \le x \le n_A)$，分配给 B 资源加暴击率的份数为整数 y $(0 \le y \le n_B)$。则最终属性为：

$$r = r_0 + a_r x + b_r y,$$
$$k = k_0 + a_k(n_A - x) + b_k(n_B - y).$$

期望伤害 $D = 1 + r(k-1)$。最大化 D 等价于最大化

$$P(x, y) = r(k-1).$$

令

$$A_0 = r_0, \quad B_0 = (k_0 - 1) + a_k n_A + b_k n_B,$$

则

$$P(x, y) = (A_0 + a_r x + b_r y)(B_0 - a_k x - b_k y).$$

枚举一维，优化另一维

由于 $n_A, n_B \le 2 \times 10^5$，我们可以枚举 x 从 0 到 n_A。固定 x 后，P 变为关于 y 的一元二次函数：

$$g(y) = (A + b_r y)(B - b_k y), \quad y \in [0, n_B] \text{ 整数},$$

其中 $A = A_0 + a_r x,\ B = B_0 - a_k x$。
展开得：

$$g(y) = -b_r b_k\, y^2 + (b_r B - b_k A)\, y + AB.$$

因为 $b_r, b_k > 0$，二次项系数为负，函数开口向下。在闭区间 $[0, n_B]$ 上的整数最大值只可能出现在以下位置：

- 左端点 $y = 0$，值 AB；
- 右端点 $y = n_B$，值 $(A + b_r n_B)(B - b_k n_B)$；
- 对称轴（实数）$y^* = \dfrac{b_r B - b_k A}{2 b_r b_k}$ 附近的整数点，即 $\lfloor y^* \rfloor$ 和 $\lceil y^* \rceil$，前提是它们落在 $[0, n_B]$ 内。

检查这些候选点（最多 4 个），取最大值即为固定 x 时的最优解。更新全局最优即可。
时间复杂度：枚举 x 需要 $O(n_A)$ 次循环，每次常数操作，总复杂度 $O(n_A)$，足以在 $n_A \le 2 \times 10^5$ 下通过。

参考代码

```
#include <bits/extc++.h>
using namespace std;
#define rep(x, y, z) for (int x = (y); x <= (z); x++)
int na, nb;
double r0, k0, ar, ak, br, bk, ans;

double cal(double a, double b, double c, double d, double e, double f, int p, int q) {
    return a * p * p + b * q * q + c * p * q + d * p + e * q + f;
}

int main() {
    cin >> r0 >> k0 >> ar >> ak >> na >> br >> bk >> nb;
    rep(q, 0, nb) {
        double c0 = r0, m0 = k0, a = -ar * ak, b = -br * bk, c = -(ar * bk + ak * br),
        d = (m0 - 1) * ar - ak * c0 + (na * ak + nb * bk) * ar,
        e = (m0 - 1) * br - bk * c0 + (na * ak + nb * bk) * br,
        f = (na * ak + nb * bk + m0 - 1) * c0 + 1;
        double p = -(c * q + d) / (2 * a);
        if (p <= 0)
        ans = max(ans, cal(a, b, c, d, e, f, 0, q));
        else if (p >= na)
        ans = max(ans, cal(a, b, c, d, e, f, na, q));
        else {
            int p1 = floor(p), p2 = ceil(p);
            ans = max(ans, cal(a, b, c, d, e, f, p1, q));
            ans = max(ans, cal(a, b, c, d, e, f, p2, q));
        }
    }
    cout << fixed << setprecision(3) << ans;
}
```

题目大意

E 春日写生

——By welken

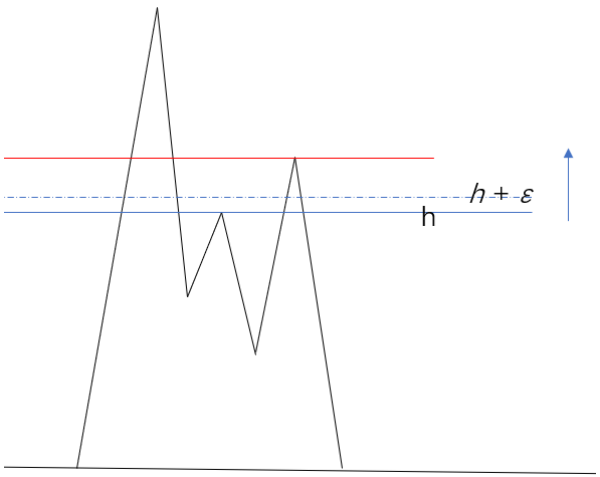
把步道看成长度 $n-1$ 的折线 $f\colon [0,n-1]\to\mathbb{R}$：第 k 个里程碑位于坐标 $k-1$、海拔 H_k，相邻里程碑之间线性相连。数据保证 $H_1=H_n=0$，$H_2,\ldots,H_{n-1}$ 是 $[1,5\times10^5]$ 内两两不同的整数，因此每段坡严格单调，步道上不会出现平地。

若 $A<B$ 满足 $f(A)=f(B)=h$，且开区间 (A,B) 内不存在海拔严格大于 h 的点，则称 (A,B) 是一对可见同高位置：等高不遮挡视线，严格更高才遮挡。

停在某点的写生风景度，是它能看见的所有同高位置中离它最远者的水平距离。求步道上所有位置的最大风景度，以最简分数输出；若不存在同高位置，则为 0。

数据范围：$1\le T\le10^4$，$3\le n\le10^5$，$\sum n\le10^5$。

最优可见对至少一端是里程碑



任取一对可见同高位置 $A < B$，高度为 h。

- 若 A 在坡内部，则 A 所在坡必须向右下降；否则紧邻 A 右侧就会出现高于 h 的点，挡住视线。
- 若 B 在坡内部，则 B 所在坡必须向右上升；否则紧邻 B 左侧就会出现高于 h 的点，挡住视线。

于是把水平视线从 h 稍微抬高到 $h + \varepsilon$：左端点会向左移动，右端点会向右移动，水平距离增大，并且中间地形仍不高于新视线，所以仍然合法。不断抬高，直到某一端到达里程碑。

结论

因此，最优可见同高对中至少有一端是里程碑。我们只需枚举每个里程碑，分别看它向左、向右能看到的最远同高位置。

第一个更高点决定最远同高点

固定里程碑 p。设 q 是 p 右侧第一个满足 $H_q > H_p$ 的里程碑。

- 若 q 不存在，则右侧到不了海拔 H_p，右侧可见距离为 0。
- 否则，(p, q) 内所有里程碑都低于 H_p。最远同高点在 $(q-1, q)$ 这一段坡上，水平距离为

$$R_p = (q - p - 1) + \frac{H_p - H_{q-1}}{H_q - H_{q-1}}.$$

左侧完全对称。设 q 是 p 左侧第一个满足 $H_q > H_p$ 的里程碑，则

$$L_p = (p - q - 1) + \frac{H_p - H_{q+1}}{H_q - H_{q+1}},$$

q 不存在时 $L_p = 0$。因此

$$\mathrm{Ans} = \max_p \max(L_p, R_p).$$

注意 $L_p + R_p$ $(L_p, R_p \neq 0)$ 也是一对可见同高位置之间的水平距离，但它不可能是最优：按第一页的抬高论证，把视线抬高能把其中一端推到某个里程碑 p'，距离严格增大，而新距离不超过 p' 那一侧的单侧值，于是 $L_p + R_p$ 恒被 $\max_{p'} \max(L_{p'}, R_{p'})$ 支配，不必单独列为候选。

实现与数值安全

- 两次单调栈求左右两侧第一个严格更高点。
- 候选分数：

$$I + \frac{N}{D} = \frac{I \cdot D + N}{D}.$$

- 比较两个候选：交叉相乘。
- 上界：

$$n(5 \times 10^5)^2 \le 2.5 \times 10^{16},$$

64 位整数足够，乘法记得用 `long long`，若存储 $L_p + R_p$ 为答案，注意交叉乘积很大，需要用 `__int128` 保存。

- 最后约分输出。

复杂度：$O(\sum n)$ 时间，$O(n)$ 空间。

题目大意

F 求王北伐

——By Lingzhang

给定一张 n 个点、m 条边的连通无向带权图，其中有 r 个驻军城市。另有 k 次询问，每次给出一个出发城市 p，需要选择一个兵力 d，使得从 p 出发，只经过边权不超过 d 的边，能够到达至少一个驻军城市。求所有询问所需 d 之和的最小值。

由于每次发兵互相独立，答案就是每次询问的最小 d 之和。

对单次询问，从 p 到某个驻军城市 s 的路径需要的最小 d，就是这条路径上的最大边权；再对所有驻军城市取最小值，即「最小瓶颈路」：

$$ans(p) = \min_{s \in S} \min_{\text{路径} P: p \to s} \max_{e \in P} w_e,$$

其中 S 为驻军城市集合。

法 1：Kruskal 重构树

到点 x 的简单路径上最大边权的最小值 $\leq val$ 的所有点 y 均在 Kruskal 重构树上的某一棵子树内，且恰好为该子树的所有叶子节点。
我们在 Kruskal 重构树上找到 x 到根的路径上权值 $\leq val$ 的最浅的节点，显然这就是所有满足条件的节点所在的子树的根节点。
对于任意两个原图节点 u, v，设 $l = \mathrm{LCA}(u, v)$，再设 e 为 l 连向「包含 u 的那个儿子」的边，则

$$\min_{\text{路径}P:u\to v} \max_{e'\in P} w_{e'} = w_e.$$

也就是说，**LCA 通往其中一方的边的权值，就是 u, v 之间的最小瓶颈值**。由于 l 到两个儿子的边权相同，取通向 u 的边或通向 v 的边都可以。

法 1：Kruskal 重构树

证明：

- 在加入权值为 w_e 的这条边之前，u, v 还不在同一个连通块，说明不存在一条只由边权小于 w_e 的边组成的 $u \to v$ 路径，因此任何路径的最大边权都不小于 w_e；
- 加入这条边之后，u, v 连通，且此时用到的边权都不超过 w_e，所以存在一条最大边权恰好为 w_e 的路径。

两者结合，最小瓶颈值就是 w_e。

1. 计算子树内驻军数量

令 $sz(i)$ 表示重构树子树 i 内驻军城市的数量。

- 驻军叶子：$sz = 1$；
- 其他叶子：$sz = 0$；
- 内部节点：$sz(i) = sz(\text{左儿子}) + sz(\text{右儿子})$。

由于新建节点的编号一定大于它的儿子，按编号从小到大遍历内部节点即可求出所有 sz。

2. 关键观察

对于非驻军叶子 p，设 j 是 p 的**最深的**、**满足** $sz(j) > 0$ **的祖先**，再设 e_j 为 j 连向「包含 p 的那个儿子」的边，则

$$ans(p) = w_{e_j}.$$

理由：

- j 的子树里一定有驻军城市。如果这个驻军城市在包含 p 的那个儿子子树里，那么那个儿子就会是更深的、满足 $sz > 0$ 的祖先，与 j 最深矛盾。所以驻军城市一定在另一个儿子子树里，于是它与 p 的 LCA 正是 j；根据瓶颈路性质，瓶颈值就是 j 连向包含 p 的儿子的那条边的边权，即 w_{e_j}。
- 对任意驻军城市 s，$\mathrm{LCA}(p, s)$ 也是 p 的、满足 $sz > 0$ 的祖先，深度不会超过 j。重构树中越靠上的合并边权越大，所以 $\mathrm{LCA}(p, s)$ 连向包含 p 的儿子的那条边的边权不小于 w_{e_j}。

因此最小值就是 w_{e_j}。

3. 自顶向下递推

设 $f(v)$ 表示：v 的最深的、满足 $sz > 0$ 的真祖先 $j(v)$ 连向包含 v 的儿子的那条边的边权。
设内部节点为 i，它的儿子为 y，边 $i \to y$ 的边权为 w（也就是 i 的合并边权），则递推为

$$f(y) = \begin{cases} w, & sz(i) > 0, \\ f(i), & sz(i) = 0. \end{cases}$$

含义：

- 如果 $sz(i) > 0$，说明 i 的子树里有驻军城市，那么 i 就是 y 的最深的满足条件的真祖先，而 $i \to y$ 的边权正是 w，所以 $f(y) = w$；
- 如果 $sz(i) = 0$，说明 i 的子树里没有任何驻军城市，i 不能作为答案来源，必须继续往上找，于是继承 $f(i)$。

由于父亲编号一定大于儿子编号，所以按编号从大到小处理内部节点，就能保证处理 i 时 $f(i)$ 已经计算完毕。
对于询问的叶子 p（题目保证不是驻军城市），上述递推得到的 $f(p)$ 就是 $w_{e_{j(p)}}$，也就是该点的最小瓶颈值。

复杂度

- **排序**：$O(m \log m)$；
- **Kruskal 与建树**：$O(m\alpha(n) + n)$；
- **统计 sz 与递推**：$O(n)$；
- **回答询问**：$O(k)$。

总时间复杂度

$$O(m \log m + n + k),$$

空间复杂度 $O(n + m)$。

法 2：Dijkstra 求最小瓶颈路

从所有驻军城市同时出发，把「路径长度」定义为「路径上的最大边权」，跑一遍 Dijkstra。得到的 $d[p]$ 就是城市 p 到任意驻军城市的最小瓶颈值，也就是该次发兵所需的最小兵力。
每次发兵互相独立。对询问城市 p：

$$ans(p) = \min_{s \in S} \min_{P: p \to s} \max_{e \in P} w_e,$$

其中 S 为驻军城市集合。
令

$$d(v) = \min_{s \in S} \min_{P: s \to v} \max_{e \in P} w_e,$$

则每次询问的答案就是 $d(p)$，总答案为

$$\sum_{i=1}^{k} d(p_i).$$

算法与复杂度

算法

1. 初始化：对所有驻军城市 s，令 $d(s) = 0$ 并放入堆；其余点的 d 为正无穷。
2. 每次取出堆中 d 最小的点 u，若取出的值大于当前的 $d(u)$，说明已过期，直接跳过。
3. 枚举 u 的邻边 (u, v, w)，松弛：

$$d(v) \leftarrow \min\Big(d(v),\ \max(d(u), w)\Big).$$

   若 $d(v)$ 被更新，就把 v 放入堆。
4. 堆为空后，累加所有询问城市的 d 值并输出。

注意：这里把普通 Dijkstra 的「加法」换成了「取 $\max$」，其余流程完全一样。

复杂度

一次多源 Dijkstra 即可回答全部询问：$O\big((n+m)\log n\big)$ 时间，$O(n+m)$ 空间，时间常数略大于 kruskal 重构树。

题目大意

G 绝趋零

——By intconstlee

给一个数组，维护区间加正值和区间求绝对值和。

解法一

先考虑如何维护区间求绝对值和，可以对正权和负权取相反数后各建立一棵线段树完成该任务。之后考虑区间加正值，相当于正权线段树上区间加，负权线段树上区间减，这里的加/减都有一个系数 size。

考虑一个负数可能会被加成正数，强制从负权线段树上删去该点加入正权线段树即可，每个点只会被转移一次，因此复杂度总和为 $O(N \log N)$，能够通过本题。

解法二

发现只需要建立一棵线段树也能够解决该问题，线段树结点上记区间和、区间负数个数、区间最大负数和懒标记。

区间加即区间和增加 $w \times (\text{len} - 2 \times \text{num of neg})$。

负数转正操作可以依靠 update 时访问到叶子更新。

题目大意

H SCUT

——By XiaoXia

在给定字符串中找出 SCUT 子序列（不区分大小写）。

做法

贪心地找到第一个出现的 S/s 的位置 i_1，再贪心地找到 i_1 之后第一个 C/c 的位置 i_2，以此类推，直到找出整个 SCUT 子序列为止。若按上述流程找不到则无解。
复杂度 $\mathcal{O}(n)$。

题目大意

I 随机加减

——By welken

给定实数序列 $a_1, a_2, \ldots, a_n$。反复执行如下操作，直到序列只剩一个元素：设当前长度为 L，在 $1 \le i < L$ 中等概率选出一个 i，再抛一枚公平硬币，以 $\frac{1}{2}$ 的概率令 $a_i \leftarrow a_i - a_{i+1}$、以 $\frac{1}{2}$ 的概率令 $a_i \leftarrow a_i + a_{i+1}$，随后删除 a_{i+1}，其后的元素整体前移。

设最终剩下的元素为 X，求 $\mathbb{E}[X]$。输出一个实数，与标准答案的绝对误差不超过 10^{-6} 即正确。

数据范围：$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$，且小数部分不超过 9 位。

题解

每次删除的是 a_{i+1}，而 $i \geq 1$，所以第一个位置永远不会被删除。最终剩下的就是第一个位置上的值，记为 X。
初始 $X = a_1$。考虑某一步操作后 X 的期望：

- 如果选中的是 $i \geq 2$，第一个位置不受影响；
- 如果选中的是 $i = 1$，则

$$X \leftarrow X + a_2 \quad \text{或} \quad X \leftarrow X - a_2$$

两种概率各为 $1/2$，所以这一步之后 X 的条件期望仍为：

$$\frac{(X + a_2) + (X - a_2)}{2} = X.$$

每操作一步，X 的期望都不变。最终：

$$\mathbb{E}[X] = a_1.$$

故答案直接输出 a_1 即可。

题目大意

J 暴铺

——By sky_sx

每个城市初始有 18 片森林、0 个建造者、0 点生产力，生产目标为空。每回合按规则选择生产目标、让建造者砍树、增加基础生产力，并在达到目标成本时完成生产。

- 移民需要 r 点生产力，完成后获得一座新城市。
- 建造者需要 s 点生产力，完成后获得一个初始体力为 3 的建造者。
- 每次砍树消耗一片森林和该建造者 1 点体力，并增加 p 点生产力。
- 每回合增加 t 点基础生产力。
- 体力为 0 的建造者消失。

给定回合数 $H \leq 60$，求最终城市数的最大值。

1. 描述一个城市的状态

算法思路：有限状态图 + 动态规划

定义状态：

$$(B, b_1, b_2, b_3, q, P)$$

其中：

- B：已经建造的建造者数量，最多记录到 18；
- b_1, b_2, b_3：剩余体力分别为 1、2、3 的建造者数量；
- q：当前生产目标，0 表示空，1 表示移民，2 表示建造者；
- P：当前生产力。

体力为 0 的建造者已经消失，不需要单独记录。

1. 状态中的森林数量

每个建造者最初有 3 点体力，因此已砍掉的森林数为：

$$C = 3B - b_1 - 2b_2 - 3b_3$$

剩余森林为：

$$F = 18 - C$$

这样就不需要额外保存森林数量。
将建造者数量限制为 18：只有 18 片森林。达到 18 个建造者后，额外建造者不会增加有用的砍树能力，可以把更大的数量合并到 18。

2. 枚举一回合内的决策

如果当前生产目标为空，可以选择移民或建造者；否则只能继续当前目标。

设目标成本为 cost，当前生产力为 P。本回合先获得基础生产力 t，剩余缺口为：

$$d = cost - (P + t)$$

如果 $d \leq 0$，不需要砍树就能完成目标，本回合砍树没有收益。

如果 $d > 0$，至少需要：

$$need = \left\lceil \frac{d}{p} \right\rceil$$

次砍树才能在本回合完成目标。

2. 枚举砍树数量

设当前活跃建造者数量为 $b_1 + b_2 + b_3$，本回合最多能砍：

$$available = \min(F, b_1 + b_2 + b_3)$$

生产目标完成后生产力会清零，因此砍树数量超过 `need` 没有意义。只需枚举：

$$0 \le k \le \min(available, need)$$

固定砍树数量 k 后，优先使用体力较高的建造者，即依次选择体力为 3、2、1 的建造者。这样在完成相同砍树量的前提下，尽量避免建造者因体力耗尽而消失，并保留后续回合的并行砍树能力。

3. 预处理状态转移图

从初始状态开始，使用 BFS 枚举最多 H 回合内能到达的所有状态，并保存：

- 状态的最短到达回合数 `depth`；
- 每个状态的所有合法转移；
- 转移是否完成了一个移民。

同一个状态只需要建立一次，因为它的后续行为完全由当前状态决定。

4. 动态规划

定义：

$$dp[h][u]$$

表示当前城市处于状态 u，还剩 h 回合时，该城市及其后代城市最终最多能有多少座城市。

初始条件：

$$dp[0][u] = 1$$

因为没有剩余回合时，只剩当前城市本身。

假设状态 u 经过一次操作转移到 v。

- 如果本回合没有完成移民，收益为 $dp[h-1][v]$；
- 如果本回合完成了移民，还会获得一个新城市。新城市有 $h-1$ 回合可用，因此额外贡献 $dp[h-1][start]$，其中 `start` 是初始城市状态。

4. 动态规划转移

因此：

$$dp[h][u] = \max_{u \to v} \left(dp[h-1][v] + \begin{cases} dp[h-1][start], & \text{完成移民} \\ 0, & \text{否则} \end{cases} \right)$$

最终答案是初始状态的 $dp[H][start]$。
状态最早在第 d 回合到达，因此只计算满足 $depth[u] + h \le H$ 的 DP 值即可。
使用滚动数组可以减少内存。

正确性证明

引理 1：状态完整。
当前城市的后续行为只取决于建造者的体力分布、剩余森林、当前生产目标和当前生产力。剩余森林可以由 B, b_1, b_2, b_3 推出，因此状态包含了所有必要信息。
引理 2：状态转移完整。
当生产目标为空时，程序枚举两种目标；砍树数量枚举所有可行值。固定砍树数量后，优先消耗高体力建造者保留尽可能多的活跃建造者。因此，所有影响最优答案的决策都被覆盖。
引理 3：动态规划转移正确。
完成移民会生成一个独立的新城市，该城市的最优贡献是 $dp[h-1][start]$。当前城市后续贡献为 $dp[h-1][v]$。两部分相加，并对所有决策取最大值，得到当前状态的最优答案。
定理：由初始状态计算出的 $dp[H][start]$，就是经过 H 回合后最多能拥有的城市数。

复杂度分析

设 BFS 得到的可达状态数为 S，状态转移数为 E。

- 时间复杂度：$O(HE)$。
- 空间复杂度：$O(S+E)$。

题目大意

K 小 Z 的小 Z

——By wsy_jim

平面上有 n 个" Z 形线"。一个 Z 形线由 2 条方向相反的平行射线 + 1 条连接端点的中间线段组成。
求绘制 n 个 Z 形线最多可以将平面分割成多少个区域。

题解

这是一个平面分割问题，我们可以通过递推分析画第 n 个 Z 形线时最多能增加多少个区域，设答案为 f_i。
我们可以画图得到 $f_0 = 1, f_1 = 2, f_2 = 12$，发现线段对平面分割的计数难以处理，不如先把线段与射线都延长成直线，计算若干直线可以把平面分成多少个区域，再减掉每个 Z 形线进行延长后会增加的区域数量，设把线段与射线都延长成直线的答案为 g_i。

题解

同样可以画图得到 $g_0 = 1, g_1 = 6$，每次画一个 Z 形线会增加 3 条直线，画了 n 条后，平面上共有 $3n$ 条直线，考虑再画一个 Z 形线：

- 首先画两条平行线，能新增出最多区域的画法就是与之前的 $3n$ 条直线都相交，那么画完这两条平行线会在原来的平面上新增 $2 \times (3n + 1)$ 个区域；
- 再画一条直线，与之前的 $3n + 2$ 条直线都相交，增加 $3n + 3$ 个平面；

也就是 $g_{n+1} = g_n + 2 \times (3n + 1) + 3n + 3 = g_n + 9n + 5$。
考虑每画一个延长后的 Z 形线会比没延长增加多少区域，画图可知，会增加 4 个区域，也就是 $f_n = g_n - 4n$。
带入上式得到 $f_{n+1} = f_n + 9n + 1$，发现这个递推式可以展开成等差数列的形式，于是能写出它的通项 $f_n = \frac{9}{2}n^2 - \frac{7}{2}n + 1$。
另：如果能够发现每增加一个 Z 形线只会增加 $O(n)$ 个区域，发现答案是二次函数的形式，也可使用待定系数求出通项。
注意计算时数值达到了 9×10^{18}，要记得使用 `long long`。

题目大意

L Cinderella

——By Kuoy

有 n 束烟花，第 i 束在时刻 a_i 开始升空。
每束烟花连续 t 秒，每秒点亮一段长度为 v 的新线段。各束烟花点亮的线段互不重叠。
在时刻 s 点亮的线段，会在时刻 $s, s+1, \ldots, s+d$ 保持点亮，并在 $s+d+1$ 时熄灭。
求所有时刻中，仍处于点亮状态的线段总长度的最大值。

观察

因为每段烟花的长度都是 v，所以只要当作是长度为 1，最后乘回 v 即可

每束烟花只需要关注产生四个关键节点：

- $(a_i, +1)$：烟花升起，即点亮的线段数量开始增加；
- $(a_i + m, -1)$：烟花开始熄灭，即点亮的线段数量停止增加；
- $(a_i + M, -1)$：烟花停止上升，即点亮的线段数量开始减少；
- $(a_i + t + d + 1, +1)$：烟花完全熄灭，即点亮的线段数量停止减少

扫描线

将所有事件按时间排序，并合并相同时刻的所有事件
使用扫描线依次处理事件
维护 k，表示每秒点亮线段数量的变化量，sum 表示当前点亮线段的总数
每经过一个事件，就将其权值累加到 k 中
由于相邻两个事件之间没有新的变化，因此这段时间内，每秒点亮线段数量的变化量始终为 k
设相邻两个事件的时间差为 Δt，则这段时间内点亮线段数量的总变化量为 $k\Delta t$，更新：$sum = sum + k\Delta t$
扫描过程中不断维护 sum 的最大值，最终答案就为 $ans = \max\{sum\} \times v$
总时间复杂度 $O(n \log n)$，空间复杂度 $O(n)$

题目大意

M xboHodx 是回文串

——By xboHodx

给定长度为 n 的小写字母串 s，从位置 1 出发，求到达位置 n 的最小总代价。

每步可以进行以下三种移动之一：

1. **右移**：从 i 到 $i+1$ $(i<n)$，代价为 x。
2. **镜像传送**：传送到任意一个对应镜像字符的位置。
   - Black 类：b $\leftrightarrow$ d，代价为 B；
   - Purple 类：p $\leftrightarrow$ q，代价为 P。
3. **同字传送**：传送到左侧或右侧最近的同字符位置，代价为 z；某侧不存在同字符时，该方向不可传送。

共有 q 次询问，每次给出镜像传送代价 P, B，独立求出从位置 1 到位置 n 的最小总代价。

思路：分层图最短路

根据数据范围：

- **不能显式建镜像边**：$\#\mathtt{b} \cdot \#\mathtt{d} + \#\mathtt{p} \cdot \#\mathtt{q}$ 最坏是 $(n/2)^2$ 级别，$n = 10^5$ 时约 2.5×10^9 条；
- **不能每个询问跑一次 Dijkstra**：$O(qn \log n)$ 直接超时。

必须把「与询问无关的部分」预处理出来。

同一类镜像传送门，在最优解里最多用 2 次。
按「Purple 用了几次 $\times$ Black 用了几次」分成 $3 \times 3 = 9$ 层，每层预处理一个与询问无关的最短路；询问时枚举 9 种组合，加上传送门费用取最小。

核心引理：每类最多用 2 次

记题面输入的两类代价为 P（*Purple*，$\mathtt{p} \leftrightarrow \mathtt{q}$）和 B（*Black*，$\mathtt{b} \leftrightarrow \mathtt{d}$）。

引理：存在一个最优走法，其中 Purple 传送至多用 2 次，Black 传送也至多用 2 次。

先看一个基本事实。只要该类传送门可用（p 和 q 都出现），那么从该类任意一个位置出发，到该类任意一个位置都至多 2 跳：

- 异字：$\mathtt{p} \to \mathtt{q}$ 或 $\mathtt{q} \to \mathtt{p}$，1 跳；
- 同字：$\mathtt{p} \to \mathtt{q} \to \mathtt{p}'$（或对称地 $\mathtt{q} \to \mathtt{p} \to \mathtt{q}'$），2 跳。

证明：
对于同一类传送，如果进行了超过 $k > 2$ 次，如 $s_1 \to s_2 \to s_3 \to \cdots \to s_k$，

- 若 s_1 与 s_k 同字：则可以删去 $s_3 \to \cdots \to s_{k-1}$，直接变成 $s_1 \to s_2 \to s_k$。
- 若 s_1 与 s_k 异字：则可以删去 $s_2 \to s_3 \to \cdots \to s_{k-1}$，直接变成 $s_1 \to s_k$。

分层最短路

令 $i, j \in \{0, 1, 2\}$，定义

$$F[i][j] = \text{恰好用} i \text{ 次 Purple、} j \text{ 次 Black 时，从位置1 到位置} n \text{ 的最小「固定花费」.}$$

「固定花费」只算右移（x）和同字传送（z），**不算**镜像传送的钱；镜像传送的钱留到询问时按 $iP + jB$ 加上。

层内
在 (i, j) 这一层里，只剩两种移动：

- $u \to u + 1$，代价 x；
- $u \to \mathrm{prev}[u]$、$u \to \mathrm{nxt}[u]$ （前/后一个同字符位置），代价 z。

这就是一张 n 个点、$O(n)$ 条边的图。对一个层的若干「种子」（已经到达的位置及其固定花费）做**多源 Dijkstra**，就得到这一层每个位置的 F 值；$F[i][j] = \mathrm{dist}[n]$。

层间
从 (i, j) 用一次 Purple 传送，会进入 $(i + 1, j)$，**且可以从任意 `p` 落到任意 `q`（或反过来）**。如果直接连边，就是 $O(n^2)$ 条，会炸。正确做法是聚合：

$$\mathrm{bestP} = \min_{s_u = \mathtt{p}} \mathrm{dist}[u],$$

$$\mathrm{bestQ} = \min_{s_u = \mathtt{q}} \mathrm{dist}[u].$$

- 每个 `q` 位置都作为 $(i + 1, j)$ 的种子，值取 bestP；
- 每个 `p` 位置都作为 $(i + 1, j)$ 的种子，值取 bestQ。

因为取到 `bestP` 的那个 `p` 本来就能一跳传到**任意** `q`，所以每个 `q` 都能拿到这个最小值。Black 完全对称：用 `bestB / bestD` 给 $(i, j + 1)$ 的 `d/b` 位置播种。

每层只花 $O(n)$ 做聚合，不需要展开任何镜像边。

分层最短路

流程

```
adj[0][0] = { (1, 0) }                    // 每层的多个起点，第(0,0)层只有节点1是起点

for i = 0..2:
    for j = 0..2:
        if adj[i][j] 为空: continue
        dist = 多源 Dijkstra(adj[i][j], 边: u->u+1 (x), u->prv/nxt (z))
        F[i][j] = dist[n]

    if i < 2:                             // 再用一次 Purple
        bestP = min dist[u] over s[u]=='p'
        bestQ = min dist[u] over s[u]=='q'
        把 (每个 q 位置, bestP) 加入 adj[i+1][j]
        把 (每个 p 位置, bestQ) 加入 adj[i+1][j]

    if j < 2:                             // 再用一次 Black
        bestB = min dist[u] over s[u]=='b'
        bestD = min dist[u] over s[u]=='d'
        把 (每个 d 位置, bestB) 加入 adj[i][j+1]
        把 (每个 b 位置, bestD) 加入 adj[i][j+1]

for 每个询问 (P, B):
    ans = min over i,j of F[i][j] + i*P + j*B
    输出 ans
```

层号可以取 `i * 3 + j`。$(i-1,j)$ 和 $(i,j-1)$ 的层号都比 (i,j) 小，所以按 i 从小到大、j 从小到大循环，天然满足拓扑序，不需要额外排序。

复杂度

9 层，每层一次 $O(n \log n)$ 的 Dijkstra 和 $O(n)$ 的聚合；每个询问枚举 9 种状态：

$$O(9n \log n + 9q) = O(n \log n + q),$$

空间 $O(n)$。9 是常数。

题目大意

N 与或异或

——By sky_sx

对于给定的 L, R，求：

$$\bigwedge_{i=L}^{R}\left(\bigvee_{j=L}^{R}\left(\bigoplus_{k=L}^{R}((i \,\&\, j) \mid (j \oplus k))\right)\right)$$

其中：

- $\&$：按位与；
- $\mid$：按位或；
- $\oplus$：按位异或。

题目允许 $R < 2^{60}$，不能枚举区间内所有整数。

1. 预处理区间异或

定义：

$$X = \bigoplus_{k=L}^{R} k$$

再定义区间长度的奇偶性：

$$q = (R - L + 1) \bmod 2$$

区间异或可以用异或前缀快速计算：

$$X = prefixXor(R) \oplus prefixXor(L - 1)$$

1. 前缀异或的周期

前缀异或具有周期 4：

$$prefixXor(n) = \begin{cases} n, & n \bmod 4 = 0 \\ 1, & n \bmod 4 = 1 \\ n+1, & n \bmod 4 = 2 \\ 0, & n \bmod 4 = 3 \end{cases}$$

因此可以在 $O(1)$ 时间求出 X。

2. 单独考虑第 b 位

设 P_b 是 X 的第 b 位，也就是区间中所有 k 的第 b 位异或结果。
对表达式：

$$(i \,\&\, j) \mid (j \oplus k)$$

只看第 b 位，有三种情况：

- 若 $j_b = 0$，表达式的这一位就是 k_b，对所有 k 异或后得到 P_b。
- 若 $j_b = 1, i_b = 1$，这一位恒为 1，异或 $N = R - L + 1$ 次后得到 q。
- 若 $j_b = 1, i_b = 0$，这一位为 $1 \oplus k_b$，异或后得到 $q \oplus P_b$。

2. 各类情况的答案

于是可以得到最终答案的每一位。

情况 A：区间内第 b 位恒为 0。

由于 L 到 R 连续，若 $L \gg b = R \gg b$，则这一位在整个区间内恒定。若该位为 0，答案的第 b 位为 0。

情况 B：区间内第 b 位恒为 1。

答案的第 b 位为 q。

情况 C：区间内第 b 位既出现 0，也出现 1。

经过对 j 的按位或，再经过对 i 的按位与，最终第 b 位为：

$$P_b \vee q$$

因此，枚举 $b = 0, 1, \ldots, 59$，根据上述规则设置答案的对应位即可。

正确性证明

引理 1

区间异或 X 的第 b 位等于所有 $k \in [L, R]$ 的第 b 位异或结果。

这是按位异或的定义直接得到的。

引理 2

对任意一位，最终答案只取决于该位在区间内是否恒为 0、是否恒为 1，以及 P_b, q。

按位与、按位或、按位异或都不会影响其他二进制位，因此可以逐位独立分析。根据 j_b 和 i_b 的取值分类，可以得到上述三种情况。

定理

算法对 60 个二进制位分别计算并合并，得到的结果与原表达式相同。

因为所有位运算均逐位独立，所有位的计算都正确，所以最终答案正确。

复杂度分析

每个查询只计算一次区间异或，并检查 60 个二进制位。

- 时间复杂度：$O(60\,T)$，其中 T 为查询次数。
- 空间复杂度：$O(1)$。

题目大意

O 十夏之花

——By XiaoXia

已知非线性方程组 $b_i = \sum_{j=1}^{n} a_j^i \pmod{998244353}, i \in [1, n]$，解出所有 $a_i \pmod{998244353}, i \in [1, n]$，并按升序输出。

多组测试，保证 $\sum n \leq 3000$，合法解存在且唯一。

题目大意

直接求解非线性方程组很困难。考虑构造一个多项式，使它的根恰好为这些未知数：

$$P(x) = \prod_{i=1}^{n} (x - a_i).$$

接下来分两步：先恢复多项式系数，再求出所有根。

由幂和恢复多项式

为了方便展开，不妨设

$$F(x) = \prod_{i=1}^{n}(1 - a_i x).$$

两边取形式幂级数对数：

$$\ln F(x) = \sum_{i=1}^{n} \ln(1 - a_i x) = -\sum_{k \geq 1} \frac{\sum_{i=1}^{n} a_i^k}{k} x^k.$$

因此只需使用已知的前 n 项幂和：

$$F(x) = \exp\left(-\sum_{k=1}^{n} \frac{b_k}{k} x^k\right) \pmod{x^{n+1}}.$$

求出 F 后，将系数反转即可得到：

$$P(x) = x^n F(x^{-1}) = \prod_{i=1}^{n}(x - a_i).$$

处理重复的根

题目没有保证 a_i 互不相同，因此必须保留根的重数。

若 r 是 P 的 m 重根，则它是 P' 的 $m-1$ 重根。因此

$$\gcd(P, P')$$

可以提取出重复因子。
代码使用平方自由分解，将多项式写成：

$$P(x) = \prod_{m \geq 1} Q_m(x)^m,$$

其中每个 Q_m 都没有重根，且不同 Q_m 互素。Q_m 中的每个根都应在答案中出现 m 次。

最后分别对每个 Q_m 求不同的根即可。这部分相当于 `https://judge.yosupo.jp/problem/polynomial_root_finding` 板子题，不再赘述。
复杂度：$\mathcal{O}(n \log n)$（常数很大）。

感谢观看

Q&A

欢迎提问与交流。